RideMetrics: Optimizing Ride-Hailing Performance Across Uber & Bolt

Project Overview

RideMetrics is a personal data analytics project based on over one year of real-world data collected while driving for Bolt and Uber in Lodz, Poland. Motivated by the need to maximize earnings while balancing academic commitments. This project uses personal trip data to answer key business questions around earnings optimization and demand patterns. Passengers' location data has been excluded due to data protection reasons. The aim is to transform raw data into a clean, structured and actionable business insight using data cleaning, exploratory analysis, time series techniques, and an interactive dashboards.

Objectives & Business Questions

To optimize my earnings by identifying the most profitable working patterns and building a machine learning model to predict the best hours and days to work- transforming over one year of personal trip data into actionable Business Insights.

S/N	Business Questions	Datasets
1	What hours of the day generates the highest earnings?	Uber
2	Which day of the week is most profitable?	Uber
3	What is weekly total earnings?	Uber
4	What is the monthly trends for over 16months	Uber
5	Which month of the year is most profitable?	Uber
6	What is the average Uber deduction rate per trip?	Uber
7	What is average earnings per km?	Uber
8	How has monthly revenue and net earnings trended for over 16months?	Uber
9	Which hours have the highest trip demand?	Uber & Bolt
10	Which days have the highest trip volume?	Uber & Bolt
11	During which seasons is demand highest?	Uber & Bolt
12	Can historical patterns predict High/Medium/Low demand periods?	Uber & Bolt
13	Can historical patterns predict High/Medium/Low earning periods?	Uber

Data Description

Data Source

The dataset was personally collected through active driving on two ride-hailing platforms — **Uber** and **Bolt** — in Lodz, Poland. The data spans over one year of real-world trip activity and was exported directly from both platform apps in CSV format.

Data Fields

Property	Details
Format	CSVs
Platforms	Bolt & Uber
Location	Lodz Poland
Coverage	Over one year (Sep 2024 - Feb 2026)

Data Fields

Uber — Trip Details Table

- global_product_name
- status
- request_timestamp_local
- begintrip_timestamp_local
- dropoff_timestamp_local
- trip_distance_miles
- trip_duration_seconds
- base_fare_local
- original_fare_local
- cancellation_fee_local

Uber — Payment Table

- City Name
- Trip UUID
- Local Amount
- Classification
- Category
- Local Timestamp

Excluded Fields

- vehicle_uuid and license_plate removed from Trip Details table for privacy reasons.
- Trip UUID and Currency Code dropped from both tables as they add no analytical value.

Joining Key

Both Uber tables will be joined on:

- request_time
- Local Timestamp

In the Silver layer to create one unified Uber trips table.

Bolt Data Fields

Bolt — Trip Details Table

- Time
- Driver Accepted
- Ride Finished
- Ride Distance
- Ride Duration
- Order State

Bolt — Payment Table

- Date
- Payment Method
- Date of Ride
- Price (no VAT)
- VAT
- Price Total

Excluded Fields

- Driver Name, Driver Phone, Driver Company, Vehicle Model and Car Reg Number removed from Trip Details table for driver privacy reasons.
- Driver App Version, Pickup Distance, Pickup Duration, Reject Reason, Estimated Pickup Time, Estimated Distance and Estimated Duration dropped as they add no analytical value.
- Country dropped as all trips are in Poland.
- Bolt Payment table loaded into Bronze layer but subsequently excluded from Silver layer after investigation revealed incomplete and discounted payment data — retained for trip volume analysis only.

Data Limitations

- No location data limits geographic and route-level analysis as passenger pickup and drop-off coordinates were excluded for GDPR compliance.
- Cancelled trips with cancellation fees are retained in all revenue reporting tables as they represent legitimate earnings but excluded from ML feature tables as zero distance and duration provide no meaningful trip characteristic signal.
- Data reflects a single driver's experience in Łódź, Poland and may not be generalised to all drivers or other cities.
- External factors such as weather, traffic conditions and local events are not captured in the dataset and may influence demand and earnings patterns.

- Bolt trip data contains incomplete and discounted payment information and has been excluded from all financial analysis. Bolt data is used solely for time-based and operational pattern analysis.
- Bolt payment table was loaded into Bronze layer with 153,314 records but subsequently excluded from the Silver layer after investigation revealed three independent data quality issues — discounted customer payments only, inconsistent payment recording times by payment method and date of ride values not corresponding to actual trip dates.
- Uber driver incentives are included in net earnings used for ML model training as they form part of actual earnings received and historically correlate with specific time periods. This approach may overestimate earnings on days when Uber no longer runs incentive programmes.
- 12 special event days were identified as statistical outliers and flagged with a binary event indicator in the ML earnings features table. These dates were verified against Łódź event records including concerts at Atlas Arena, cultural events and public holidays.

Data Quality Findings

Methodology

RideMetrics follows a structured analytics workflow built around the **Medallion Architecture** in Microsoft SQL Server, progressing data from raw ingestion through to business-ready insights.

Medallion Architecture (SQL Layer)

Layer	Purpose	Tables
1. Bronze	Raw CSV data loaded as-is from Uber & Bolt	Bronze.Uber_Trips, Bronze.Uber_Payments, Bronze.Bolt_Trips, Bronze.Bolt_Payments
2. Silver	Cleaned, standardized and merged trip data with engineered features	Silver.Uber, Silver.Uber_Payments_Silver_pivoted, Silver.Bolt, Silver.Combined_Silver
3. Gold	Aggregated business-ready tables feeding Power BI and ML model	Gold.Daily_Revenue, Gold.Hourly_Revenue, Gold.Weekly_Revenue, Gold.Monthly_Revenue, Gold.Yearly_Monthly_Revenue, Gold.Day_of_Week_Revenue, Gold.Hourly_Demand, Gold.Daily_Demand, Gold.Seasonal_Demand, Gold.ML_Demand, Gold.ML_Earnings

Machine Learning

RideMetrics adopted two machine learning models, each addressing a distinct business question. Three algorithms were trained and compared for each model to ensure the best performing model was selected. XGBoost was selected as the best model for demand prediction achieving 96.5% accuracy while Logistic Regression was selected as the best model for earnings prediction achieving 77.0% accuracy.

Algorithm	Role	Demand Model	Earnings Model
Logistic Regression	Baseline model	73.1%	77.0% ✓
Random Forest Classifier	Primary model	78.9%	75.1%
XGBoost	Advanced benchmark	96.5% ✓	74.1%

Model 1: Demand Pattern Prediction

This model predicts whether a given hour and day combination represents a High, Medium, or Low demand period based on historical trip volume patterns. Both Uber and Bolt datasets were used since demand prediction relies on trip volume and time-based features rather than fare values. Demand categories were defined using a scoring system combining hour of day, day of week and season — reflecting patterns identified during EDA.

Target Variable: Demand category — High / Medium / Low

Input Features: hour_minutes, month, day of week (OHE), season (OHE), period_Morning.

Use Case: *"Is it worth going online right now based on expected trip demand?"*

Best Algorithm: XGBoost

Results: Accuracy 96.5% — F1 Score 96.3% — CV Mean F1 96.5% (Std 0.63%)

Conclusion: Conclusion: XGBoost achieved 96.5% accuracy in predicting High/Medium/Low demand periods after SHAP-based feature selection and One Hot Encoding were applied. The model perfectly identifies all High demand periods ensuring the driver never misses peak earning opportunities. Low demand periods show slight underperformance which is an acceptable trade-off as misclassifying a Low period as Medium results in minimal loss compared to missing a High demand period. The cross validation mean F1 of 96.5% with a standard deviation of only 0.63% confirms the model is extremely stable and generalises well to unseen data.

Model 2: Earnings Pattern Prediction

This model predicts whether a given hour represents a High, Medium, or Low earning period based on historical hourly net earnings patterns. Only Uber data was used since Bolt payment data is excluded from all financial analysis. Earnings were aggregated at hourly level as the goal is to predict how much can be earned per hour rather than per trip. Thresholds were defined using the 33rd and 66th percentiles — Low below 26.14 PLN per hour, Medium between 26.14 and 41.81 PLN per hour and High above 41.81 PLN per hour.

Target Variable: Earnings category — High / Medium / Low

Input Features: total_trips, avg_distance_km, avg_duration_minutes, demand_score, period_morning, period_evening.

Use Case: *"Is it worth going online right now based on expected earnings?"*

Best Algorithm: Logistic Regression

Results: Accuracy 77.0% — F1 Score 77.2% — CV Mean F1 78.9% (Std 1.96%)

Conclusion: Logistic Regression achieved 77.0% accuracy in predicting High/Medium/Low earning periods after SHAP-based feature selection reduced features from 19 to 6. The model reliably identifies the most profitable hours with High earning periods showing the strongest performance. Medium earning periods show the weakest performance as they sit between High and Low thresholds making them naturally harder to classify. The cross validation mean F1 of 78.9% with standard deviation of 1.96% confirms the model is stable and consistent across different data splits. The moderate overall accuracy reflects the inherent unpredictability of per-hour earnings driven by surge pricing and passenger destination factors not captured in the dataset.

Combined Model Output

The true value of both models lies in combining their predictions into a single shift recommendation. By running both models simultaneously a driver can make a fully informed decision about whether to go online at any given hour:

- **High Demand + High Earnings** → Definitely work — peak period with maximum returns
- **High Demand + Low Earnings** → Work for trip volume — busy period but lower individual fares
- **Low Demand + High Earnings** → Work for quality trips — fewer trips but higher value per trip
- **Low Demand + Low Earnings** → Stay home — not worth going online

Analysis Phases

Phase	Description	Tools
1. Data Ingestion	Load Raw CSVs into Bronze layers	Python (Pandas, pyodbc), MSSQL
2. Data Cleaning and Feature Engineering	Standardize, merge, and enrich data in Silver layer	Python (pandas), MSSQL
3. EDA	Uncover patterns in earnings, demand, and trip volume across time periods, days, months and seasons	Python (Matplotlib, Seaborn)
4. Machine Learning	XGBoost Classifier selected for Demand Prediction (Uber + Bolt) achieving 96.5% accuracy after SHAP feature optimization. Logistic Regression selected for Earnings Prediction (Uber only) achieving 77.0% accuracy after SHAP feature optimization. One Hot Encoding applied to all categorical features. Random Forest and XGBoost used as benchmarks for both models.	Python (Scikit-Learn, XGBoost)
5. Dashboard	Interactive visualizations connected directly to Gold layer tables in MSSQL	POWER BI, MSSQL
6. Documentation & Portfolio	Project README	GITHUB

Project Timing

The project was executed across six phases covering data ingestion, data cleaning, exploratory data analysis, machine learning, dashboard development, and documentation. For a detailed breakdown of each phase see RideMetrics_Timing — Project Timeline

Exploratory Data Analysis (EDA)

A detailed EDA mapping table tracking 12 business questions, chart types, findings and status was maintained separately. All 12 business questions have been answered through visualizations covering earnings optimization, demand patterns, platform comparison and time series analysis. For full findings see RideMetrics - EDA Mapping.

Tools & Technologies

Programming & Data Processing

- Python — primary programming language for data ingestion, cleaning, feature engineering, EDA and machine learning
- pandas — data manipulation and transformation
- pyodbc — connecting Python to MSSQL for data ingestion
- matplotlib & seaborn — exploratory data visualizations
- scikit-learn — building and evaluating Logistic Regression and Random Forest models
- XGBoost — advanced gradient boosting model used for demand prediction

Database & Data Engineering

- Microsoft SQL Server (MSSQL) — primary database for implementing the Medallion Architecture (Bronze, Silver, Gold layers)
- SQL — querying, transforming and aggregating data across all three layers
- Excel — data exploration and validation including commission calculations, consistency checks and SUMPRODUCT analysis during the data quality investigation phase

Business Intelligence & Visualization

- Power BI — interactive dashboard connected directly to Gold layer tables in MSSQL

Development & Collaboration

- Jupyter Notebook — Python development environment for EDA and machine learning
- GitHub — version control and portfolio hosting
- Notion — project planning and documentation

RideMetrics_Timing

RideMetrics - EDA Mapping